

## Módulo 10: Pruebas y TDD

### Contenidos clave

**pytest: fixtures, parametrización, markers:** Pytest es una herramienta para crear y ejecutar pruebas en Python, permite comprobar automáticamente si una función produce el resultado esperado.

- **Fixtures:** una fixture prepara datos o recursos que necesitan nuestras pruebas, tiene la ventaja de que podemos reutilizar en muchas pruebas.
- **Parametrización:** permite ejecutar la misma prueba con diferentes datos. “En lugar de escribir tres pruebas diferentes, se escribe una sola”.
- **Markers:** sirven para clasificar pruebas, son útiles cuando se tienen muchas pruebas.

**Mocking con unittest.mock:** Un mock es un objeto falso que se utiliza para simular una dependencia, un ejemplo es que nuestro código llama a una API externa, pero no queremos eso realmente durante cada prueba, entonces se simula, Mock(), crea una dependencia falsa que nos devuelve lo esperado. Sirve para probar el código sin depender de cosas externas, como: APIs, bases de datos, servicios de correo, archivos y servicios externos.

**Property-based testing con Hypothesis:** Hypothesis permite probar una función con muchísimos valores generados automáticamente. En una prueba normal nosotros elegimos los datos, pero con Hypothesis es como si dijéramos “No me interesa un valor específico; quiero comprobar que esta propiedad se cumpla”. La idea es comprobar que la propiedad se cumple para una gran variedad de entradas.

**Cobertura e integración en CI:** La cobertura indica qué porcentaje de nuestro código está siendo ejecutado por las pruebas, aunque el que se muestre un 100% de cobertura, no significa automáticamente que el código esté perfectamente probado; se puede ejecutar una línea sin comprobar correctamente su comportamiento.

- **TDD - Test Driven Development:** Desarrollo dirigido por pruebas. La idea principal es: Se escribe la prueba -> la prueba falla -> se escribe el código -> la prueba pasa -> se mejora/refactoriza el código. RED -> GREEN -> REFACTOR.

Finalmente, estas pruebas pueden ejecutarse automáticamente en CI, con GitHub Actions.

Developer -> git push -> GitHub -> GitHub Actions -> pytest -> Coverage -> Ruff / Black / Mypy - Todo ok.

Así, cada vez que se hacen cambios, el proyecto puede comprobar automáticamente que se sigue teniendo una suite estable.

### Objetivos

**- Practicar TDD y diseñar pruebas confiables.**

Aplicación de Red -> Green -> Refactor:

Primero se creó una prueba para cancelar órdenes PENDING, se verificó que fallara ya que el endpoint aún no existía:

```
(python-mastery-capstone-py3.14) PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> poetry run pytest .\labs\day09_fast_api\tests\test_api.py -k cancel_pending_order -q
F [100%]
===== FAILURES =====
_____ test_cancel_pending_order _____

client = <starlette.testclient.TestClient object at 0x0000020EEDB997F0>

    def test_cancel_pending_order(client):
        token = get_token(client)

        headers = {
            "Authorization": f"Bearer {token}",
        }

        order = create_order(
            client,
            headers,
        )

        response = client.post(
            f"/orders/{order['id']}/cancel",
            headers=headers,
        )

>       assert response.status_code == 200
E       assert 404 == 200
E       + where 404 = <Response [404 Not Found]>.status_code

labs\day09_fast_api\tests\test_api.py:166: AssertionError
===== warnings summary =====
..\..\..\AppData\Local\pypoetry\Cache\virtualenvs\python-mastery-capstone-Sh3whZ1x-py3.14\Lib\site-packages\_pytest\config\_init__.py:1464
  C:\Users\Quiryat\AppData\Local\pypoetry\Cache\virtualenvs\python-mastery-capstone-Sh3whZ1x-py3.14\Lib\site-packages\_pytest\config\_init__.py:1464: PytestConfigWarning: Unknown config option: asyncio_mode

    self._warn_or_fail_if_strict(f"Unknown config option: {key}\n")

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
===== short test summary info =====
FAILED labs\day09_fast_api\tests\test_api.py::test_cancel_pending_order - assert 404 == 200
1 failed, 11 deselected, 1 warning in 0.41s
```

Se implementó la funcionalidad mínima y la prueba pasó:

```
(python-mastery-capstone-py3.14) PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> poetry run pytest .\labs\day09_fast_api\tests\test_api.py -k cancel_pending_order -q
. [100%]
===== warnings summary =====
..\..\..\AppData\Local\pypoetry\Cache\virtualenvs\python-mastery-capstone-Sh3whZ1x-py3.14\Lib\site-packages\_pytest\config\_init__.py:1464
  C:\Users\Quiryat\AppData\Local\pypoetry\Cache\virtualenvs\python-mastery-capstone-Sh3whZ1x-py3.14\Lib\site-packages\_pytest\config\_init__.py:1464: PytestConfigWarning: Unknown config option: asyncio_mode

    self._warn_or_fail_if_strict(f"Unknown config option: {key}\n")

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
1 passed, 11 deselected, 1 warning in 0.31s
(python-mastery-capstone-py3.14) PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone>
```

Se agregaron reglas de negocio y pruebas parametrizadas:

```
===== short test summary info =====
FAILED labs\day09_fast_api\tests\test_api.py::test_cancel_order_with_non_pending_status[CONFIRMED] - assert 200 == 409
FAILED labs\day09_fast_api\tests\test_api.py::test_cancel_order_with_non_pending_status[SHIPPED] - assert 200 == 409
FAILED labs\day09_fast_api\tests\test_api.py::test_cancel_order_with_non_pending_status[CANCELLED] - assert 200 == 409
3 failed, 12 deselected, 1 warning in 0.70s
(python-mastery-capstone-py3.14) PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone>
```

Se incorporó mocking con unittest.mock para verificar las notificaciones y se validaron casos exitosos, inválidos, autenticación y aislamiento entre usuarios:

```
(python-mastery-capstone-py3.14) PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> poetry run pytest .\labs\day09_fast_api\tests\test_api.py -k cancel -q
..... [100%]
===== warnings summary =====
..\..\AppData\Local\pypoetry\Cache\virtualenvs\python-mastery-capstone-Sh3whZ1x-py3.14\Lib\site-packages\_pytest\config\_init_.py:1464
  C:\Users\Quiryat\AppData\Local\pypoetry\Cache\virtualenvs\python-mastery-capstone-Sh3whZ1x-py3.14\Lib\site-packages\_pytest\config\_init_.py:1464: PytestConfigWarning: Unknown config option: asyncio_mode

    self._warn_or_fail_if_strict(f"Unknown config option: {key}\n")

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
4 passed, 11 deselected, 1 warning in 0.70s
(python-mastery-capstone-py3.14) PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone>
```

### - Asegurar cobertura suficiente y suite estable.

Se creó una suite automatizada con:

- Fixtures reutilizables
- Base de datos SQLite temporal
- Pruebas parametrizadas
- Mocking
- Validación con Hypothesis

```
(python-mastery-capstone-py3.14) PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> poetry run pytest .\labs\day09_fast_api\tests\test_properties.py -q
. [100%]
1 passed in 1.86s
(python-mastery-capstone-py3.14) PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone>
```

- Ejecución automática mediante GitHub Actions

La suite alcanzó:

```
(python-mastery-capstone-py3.14) PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> poetry run pytest .\labs\day09_fast_api\tests
>> --cov=labs.day09_fast_api
>> --cov-report=term-missing
>> --cov-fail-under=80
===== test session starts =====
platform win32 -- Python 3.14.6, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\Quiryat\Documents\Python\python-mastery-capstone
configfile: pyproject.toml
plugins: anyio-4.15.1, hypothesis-6.168.1, asyncio-1.4.0, cov-7.1.0
asyncio: mode=Mode.AUTO, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_defau
lt test_loop_scope=function
collected 20 items

labs\day09_fast_api\tests\test_api.py ..... [ 95%]
labs\day09_fast_api\tests\test_properties.py . [100%]

===== tests coverage =====
----- coverage: platform win32, python 3.14.6-final-0 -----
Name                                                    Stmts  Miss  Cover   Missing
-----
labs\day09_fast_api\__init__.py                        0      0  100%
labs\day09_fast_api\auth.py                            20      4   80%   48-55
labs\day09_fast_api\database.py                       10      4   60%   19-24
labs\day09_fast_api\dependencies.py                   22      4   82%   38-41, 46
labs\day09_fast_api\init_db.py                        11     11    0%   1-23
labs\day09_fast_api\main.py                           19      0  100%
labs\day09_fast_api\models.py                         31      0  100%
labs\day09_fast_api\routers\__init__.py               0      0  100%
labs\day09_fast_api\routers\auth.py                   28      0  100%
labs\day09_fast_api\routers\orders.py                 61      3   95%   138, 163, 192
labs\day09_fast_api\schemas.py                       27      0  100%
labs\day09_fast_api\tests\__init__.py                 0      0  100%
labs\day09_fast_api\tests\conftest.py                  24      0  100%
labs\day09_fast_api\tests\test_api.py                 125     0  100%
labs\day09_fast_api\tests\test_properties.py           10      0  100%
-----
TOTAL                                                    388     26   93%
Required test coverage of 80% reached. Total coverage: 93.30%
===== 20 passed in 4.19s =====
(python-mastery-capstone-py3.14) PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone>
```

## Laboratorio

**- Implementar una historia nueva con TDD; añadir test de propiedades; reporte de cobertura.**

Historia nueva con TDD:

Se implementó: POST /orders/{order\_id}/cancel

La orden solo puede cancelarse cuando está en estado PENDING. También se validaron órdenes existentes, estados no permitidos y acceso sin autenticación.

Se utilizó Hypothesis para generar automáticamente distintos valores válidos de:

- Nombre de producto
- Cantidad
- Precio

```
(python-mastery-capstone-py3.14) PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> poetry run pytest .\labs\day09_fast_api\tests\test_properties.py -q
. [100%]
1 passed in 1.86s
(python-mastery-capstone-py3.14) PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone>
```

El reporte de cobertura se generó mediante pytest-cov con 93% de cobertura:

Coverage report: 93%

filter...

filter...

Files

Functions

Classes

hide covered

coverage.py v7.16.1, created at 2026-09-24 13:46 -0600

| File                                    | statements | missing | excluded | coverage |
|-----------------------------------------|------------|---------|----------|----------|
| labs\day09_fast_api\__init__.py         | 0          | 0       | 0        | 100%     |
| labs\day09_fast_api\auth.py             | 20         | 4       | 0        | 80%      |
| labs\day09_fast_api\database.py         | 10         | 4       | 0        | 60%      |
| labs\day09_fast_api\dependencies.py     | 22         | 4       | 0        | 82%      |
| labs\day09_fast_api\init_db.py          | 11         | 11      | 0        | 0%       |
| labs\day09_fast_api\main.py             | 19         | 0       | 0        | 100%     |
| labs\day09_fast_api\models.py           | 31         | 0       | 0        | 100%     |
| labs\day09_fast_api\routers\__init__.py | 0          | 0       | 0        | 100%     |
| labs\day09_fast_api\routers\auth.py     | 28         | 0       | 0        | 100%     |
| labs\day09_fast_api\routers\orders.py   | 61         | 3       | 0        | 95%      |
| labs\day09_fast_api\schemas.py          | 27         | 0       | 0        | 100%     |
| labs\day09_fast_api\tests\__init__.py   | 0          | 0       | 0        | 100%     |
| labs\day09_fast_api\tests\conftest.py   | 24         | 0       | 0        | 100%     |
| labs\day09_fast_api\tests\test_api.py   | 125        | 0       | 0        | 100%     |
| Total                                   | 378        | 26      | 0        | 93%      |

coverage.py v7.16.1, created at 2026-09-24 13:46 -0600

```
(python-mastery-capstone-py3.14) PS C:\Users\Quiryat\Documents\Python\python-mastery-capstone> poetry run pytest .\labs\day09_fast_api\tests --cov=labs.day09_fast_api --cov-report=term-missing --cov-fail-under=80
===== test session starts =====
platform win32 -- Python 3.14.6, pytest-9.1.1, pluggy-1.6.0
rootdir: C:\Users\Quiryat\Documents\Python\python-mastery-capstone
configfile: pyproject.toml
plugins: anyio-4.15.1, hypothesis-6.168.1, asyncio-1.4.0, cov-7.1.0
asyncio: mode=Mode.AUTO, debug=False, asyncio_default_fixture_loop_scope=None, asyncio_default_test_loop_scope=function
collected 19 items

labs\day09_fast_api\tests\test_api.py ..... [100%]

===== tests coverage =====
----- coverage: platform win32, python 3.14.6-final-0 -----
Name                               Stmts  Miss  Cover   Missing
-----
labs\day09_fast_api\__init__.py      0      0  100%
labs\day09_fast_api\auth.py          20      4   80%  48-55
labs\day09_fast_api\database.py      10      4   60%  19-24
labs\day09_fast_api\dependencies.py  22      4   82%  38-41, 46
labs\day09_fast_api\init_db.py       11     11    0%  1-23
labs\day09_fast_api\main.py          19      0  100%
labs\day09_fast_api\models.py        31      0  100%
labs\day09_fast_api\routers\__init__.py 0      0  100%
labs\day09_fast_api\routers\auth.py  28      0  100%
labs\day09_fast_api\routers\orders.py 61      3   95%  138, 163, 192
labs\day09_fast_api\schemas.py      27      0  100%
labs\day09_fast_api\tests\__init__.py 0      0  100%
labs\day09_fast_api\tests\conftest.py 24      0  100%
labs\day09_fast_api\tests\test_api.py 125     0  100%
-----
TOTAL                               378     26   93%
Required test coverage of 80% reached. Total coverage: 93.12%
===== 19 passed in 4.86s =====
```

Katherine Quiryat Rosales Garduño

Y se configuró el workflow de GitHub Actions para exigir al menos el 80% de cobertura, el módulo quedó validado localmente y mediante CI.